



BM Công nghệ phần mềm – Khoa CNTT
se.huce.edu.vn

CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN NÂNG CAO

Chương 4. Sắp xếp

Chương 4. Sắp xếp

4.1. Giới thiệu

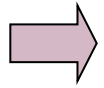
4.2. Selection Sort

4.3. Heap Sort

4.4. Quick Sort

4.5. Merge Sort

Chương 4. Sắp xếp



4.1. Giới thiệu

4.2. Selection Sort

4.3. Heap Sort

4.4. Quick Sort

4.5. Merge Sort

4.1. Giới thiệu

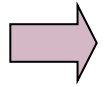
- Bài toán sắp xếp: Sắp xếp là quá trình xử lý một danh sách các phần tử để đặt chúng theo một thứ tự thỏa yêu cầu cho trước
- Ví dụ: danh sách trước khi sắp xếp:
 $\{1, 25, 6, 5, 2, 37, 40\}$
 - Danh sách sau khi sắp xếp:
 $\{1, 2, 5, 6, 25, 37, 40\}$
- Thông thường, sắp xếp giúp cho việc tìm kiếm được nhanh hơn.

4.1. Giới thiệu

- Các phương pháp sắp xếp thông dụng:
 - Buble Sort
 - Selection Sort
 - Insertion Sort
 - Quick Sort
 - Merge Sort
 - Heap Sort
 - Radix Sort

Chương 4. Sắp xếp

4.1. Giới thiệu



4.2. Selection Sort

4.3. Heap Sort

4.4. Merge Sort

4.5. Quick Sort

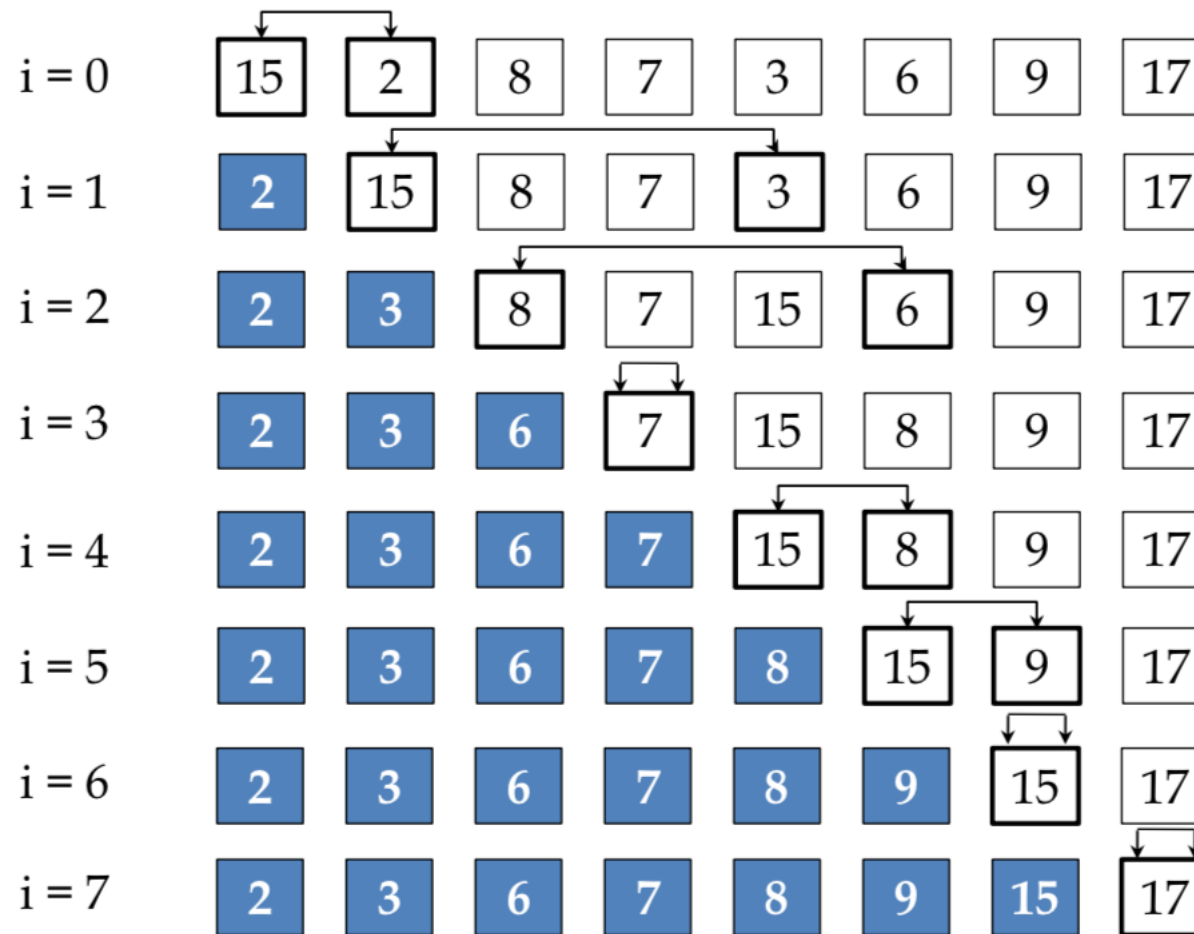
4.2. Selection Sort

- Ý tưởng
 - Chọn phần tử nhỏ nhất và đưa về vị trí đúng là đầu dãy hiện hành.
 - Sau đó xem dãy hiện hành chỉ còn $n-1$ phần tử.
 - Lặp lại cho đến khi dãy hiện hành chỉ còn 1 phần tử.

4.2. Selection Sort

- Các bước thuật toán
 - Bước 1. Khởi gán $i = 0$.
 - Bước 2. Bước lặp:
 - 2.1. Tìm $a[\text{min}]$ nhỏ nhất trong dãy từ $a[i]$ đến $a[n-1]$
 - 2.2. Hoán vị $a[\text{min}]$ và $a[i]$
 - Bước 3. So sánh i và n :
 - Nếu $i \leq n$ thì tăng i thêm 1 và lặp lại bước 2.
 - Ngược lại: Dừng thuật toán.

4.2. Selection Sort



4.2. Selection Sort

- **Đánh giá giải thuật:**

- **Số phép so sánh:**

- Tại lượt i bao giờ cũng cần $(n-i-1)$ số lần so sánh
 - Thông phụ thuộc vào tình trạng dãy số ban đầu

- Số phép so sánh = $\sum_{i=0}^{n-1} (n - i - 1) = \frac{n(n-1)}{2}$

4.2. Selection Sort

- Đánh giá giải thuật:

- Số phép so sánh:

- Tại lượt i bao giờ cũng cần $(n-i-1)$ số lần so sánh
 - Thông phụ thuộc vào tình trạng dãy số ban đầu

- Số phép so sánh = $\sum_{i=0}^{n-1} (n - i - 1) = \frac{n(n-1)}{2}$

Chương 4. Sắp xếp

4.1. Giới thiệu

4.2. Selection Sort

→ 4.3. Heap Sort

4.4. Quick Sort

4.5. Merge Sort

4.3. Sắp xếp vun đống

- Ý tưởng: khi tìm phần tử nhỏ nhất ở bước i , phương pháp Selection sort không tận dụng được các thông tin đã có nhờ vào các phép so sánh ở bước $i-1$ -> cần khắc phục nhược điểm này.
- J. Williams đã đề xuất phương pháp sắp xếp Heapsort.
- Định nghĩa Heap : Giả sử xét trường hợp sắp xếp tăng dần, Heap được định nghĩa là một dãy các phần tử $a_l, a_{l+1}, \dots, a_r$ thỏa: với mọi i thuộc $[l, r]$ (chỉ số bắt đầu từ 0)
 - $a_i \geq a_{2i+1}$
 - $a_i \geq a_{2i+2}$ $\{(a_i, a_{2i+1}), (a_i, a_{2i+2})$ là các cặp phần tử liên đới}

4.3. Sắp xếp vun đống

- Các tính chất của Heap:
 - Nếu $a_1, a_{1+1}, \dots, a_r$ là một heap thì phần tử a_1 (đầu heap) luôn là phần tử lớn nhất.
 - Mọi dãy $a_i, a_{i+1}, \dots, a_r$ với $2i + 1 > r$ là heap.

4.3. Sắp xếp vun đống

- Thuật toán
 - Giai đoạn 1: Hiệu chỉnh dãy ban đầu thành heap (bắt đầu từ phần tử giữa của dãy)
 - Giai đoạn 2: sắp xếp dựa trên heap.
 - Bước 1: đưa phần tử lớn nhất về vị trí đúng ở cuối dãy
 - Bước 2:
 - Loại bỏ phần tử lớn nhất ra khỏi heap: $r = r - 1$
 - Hiệu chỉnh lại phần còn lại của dãy.
 - Bước 3: So sánh r và l :
 - Nếu $r > l$ thì lặp lại bước 2.
 - Ngược lại, dừng thuật toán.

4.3. Sắp xếp vun đống

- Mã giả (Tựa ngôn ngữ lập trình C):

```
void HeapSort(int a[], int n)
{
    TaoHeap(a,n-1);
    r = n-1;
    while(r > 0) {
        HoanVi(a[0], a[r]);
        r = r - 1;
        HieuChinh(a,0,r);
    }
}
```


4.3. Sắp xếp vun đống

- Mã giả (Tựa ngôn ngữ lập trình C):

```
void TaoHeap(int a[], int r) {  
    int l = r/2;  
    while(l > 0) {  
        HieuChinh(a,l,r);  
        l = l - 1;  
    }  
}
```

4.3. Sắp xếp vun đống

- Mã giả (Tựa ngôn ngữ lập trình C):

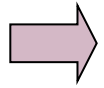
```
void HieuChinh(int a[], int l, int r)
{
    i = l; j = 2*i+1; x = a[i];
    while(j <= r)
    {
        if(có đủ 2 phần tử liên đới)
            //xác định phần tử liên đới lớn nhất
        if(a[j] < x) //thỏa quan hệ liên đới //dừng
        else
            // Hiệu chỉnh
            //Xét khả năng hiệu chỉnh lan truyền
    }
}
```

Chương 4. Sắp xếp

4.1. Giới thiệu

4.2. Selection Sort

4.3. Heap Sort



4.4. Quick Sort

4.5. Merge Sort

4.4. Sắp xếp nhanh

- Giải thuật: dựa trên việc phân hoạch dãy ban đầu thành 2 phần:

- Dãy con 1: $a_0, a_1, \dots, a_i$ có giá trị nhỏ hơn x
- Dãy con 2: $a_j, \dots, a_{n-1}$ có giá trị lớn hơn x .
- Dãy ban đầu được phân thành 3 phần:

$a_k < x$	$a_k = x$	$a_k > x$
$k = 0 \dots i$	$k = i+1 \dots j$	$k = j+1, \dots n-1$

- Phần 2 đã có thứ tự
- Phần 1, 3: cần sắp thứ tự, tiến hành phân hoạch từng dãy con theo cách phân hoạch dãy ban đầu

4.4. Sắp xếp nhanh

- Thuật toán – Giai đoạn phân hoạch

1. Chọn phần tử $a[k]$ trong dãy là giá trị mốc, $0 \leq k \leq r-1$:

- Gán $x = a[k]$, $i = l$, $j = r$.
- Thường chọn phần tử ở giữa dãy: $k = (l+r)/2$

2. Phát hiện và hiệu chỉnh cặp phần tử $a[i]$, $a[j]$ sai vị trí:

- Trong khi $(a[i] < x)$, tăng i .
- Trong khi $(a[j] > x)$, giảm j .
- Nếu $i \leq j$ thì:
 - Hoán vị $a[i]$, $a[j]$
 - Tăng i và giảm j

3. So sánh i và j :

- Nếu $i < j$: lặp lại bước 2
- Ngược lại: dừng phân hoạch.

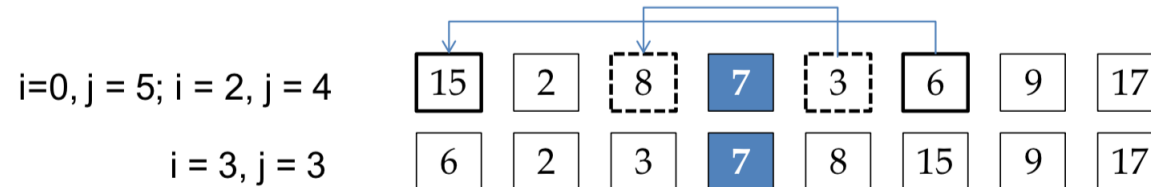
4.4. Sắp xếp nhanh

- Thuật toán Quick Sort
 - Bước 1: phân hoạch dãy ban đầu thành 3 dãy:
 - Dãy 1: $a_l \dots a_j < x$
 - Dãy 2: $a_{j+1} \dots a_{i-1} = x$
 - Dãy 3: $a_i \dots a_r > x$
 - Bước 2: sắp xếp:
 - Nếu $l < j$: phân hoạch dãy $a_l \dots a_j$
 - Nếu $i < r$: phân hoạch dãy $a_i \dots a_r$

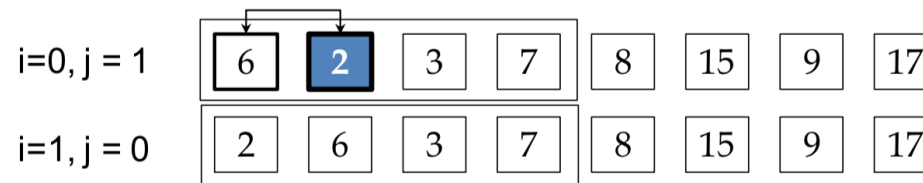
4.4. Sắp xếp nhanh

- Ví dụ

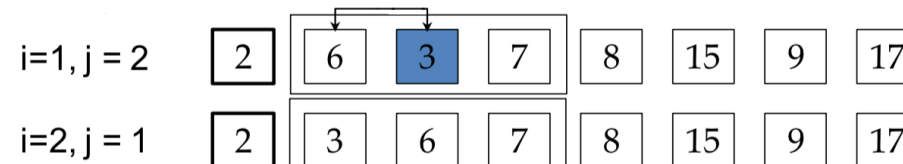
- Phân hoạch dãy ban đầu: $l = 0, r = 7, x = a[3]$



- Phân hoạch đoạn $l = 0, r = 3, x = a[1]$

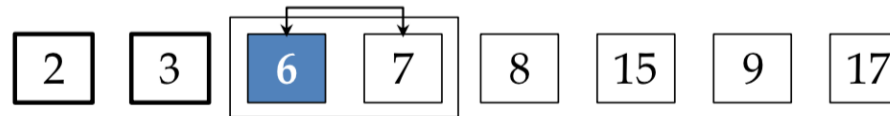


- Phân hoạch đoạn $l = 1, r = 3, x = a[2]$



4.4. Sắp xếp nhanh

- Phân hoạch đoạn $l=2$, $r=3$, $x=a[2]$



- Sinh viên tự thực hiện nốt

4.4. Sắp xếp nhanh

- Bài tập
 - Chạy tay thuật toán Quick Sort để sắp xếp mảng A trong 2 trường hợp tăng dần và giảm dần $A = \{2, 9, 5, 12, 20, 15, -8, 10\}$
- 2 9 5 12 20 15 -8 10
- $i=3, j=7 \Rightarrow$ 2 9 5 10 20 15 -8 12
- $i=4, j=6 \Rightarrow$ 2 9 5 10 -8 15 20 12
- $i=5, j=5 \Rightarrow$
- $0-5 \Rightarrow$ 2 9 5 10 -8 15

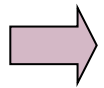
Chương 4. Sắp xếp

4.1. Giới thiệu

4.2. Selection Sort

4.3. Heap Sort

4.4. Quick Sort



4.5. Merge Sort

4.5. Sắp xếp trộn

- Thực hiện theo hướng chia để trị.
- Do John von Neumann đề xuất năm 1945.

4.5. Sắp xếp trộn

- Giải thuật

- Nếu dãy có chiều dài là 0 hoặc 1: đã được sắp xếp.
- Ngược lại:
 - Chia dãy thành 2 dãy con (chiều dài tương đương nhau).
 - Sắp xếp trên từng dãy con bằng thuật toán Merge Sort.
 - Trộn 2 dãy con (đã được sắp xếp) thành một dãy mới đã được sắp xếp.
- Input: Dãy A và các chỉ số left, right (sắp xếp dãy A gồm các phần tử có chỉ số từ left đến right).
- Output: Dãy A đã được sắp xếp

```
MergeSort(A, left, right)
{
    if (left < right) {
        mid = (left + right)/2;
        MergeSort(A, left, mid);
        MergeSort(A, mid+1, right);
        Merge(A, left, mid, right);
    }
}
```

4.5. Sắp xếp trộn

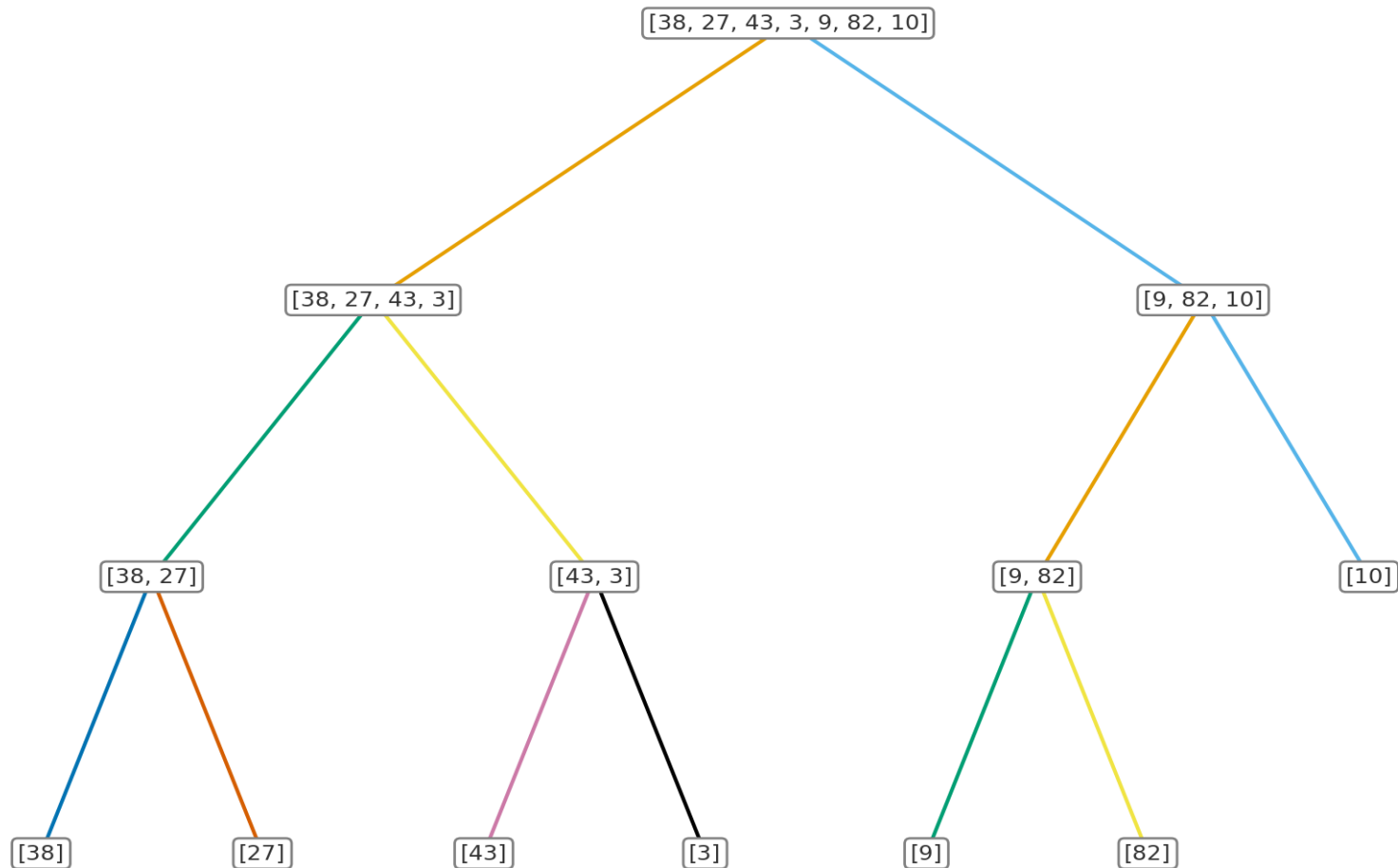
- Giải thuật

- Nếu dãy có chiều dài là 0 hoặc 1: đã được sắp xếp.
- Ngược lại:
 - Chia dãy thành 2 dãy con (chiều dài tương đương nhau).
 - Sắp xếp trên từng dãy con bằng thuật toán Merge Sort.
 - Trộn 2 dãy con (đã được sắp xếp) thành một dãy mới đã được sắp xếp.
- Input: Dãy A và các chỉ số left, right (sắp xếp dãy A gồm các phần tử có chỉ số từ left đến right).
- Output: Dãy A đã được sắp xếp

```
MergeSort(A, left, right)
{
    if (left < right) {
        mid = (left + right)/2;
        MergeSort(A, left, mid);
        MergeSort(A, mid+1, right);
        Merge(A, left, mid, right);
    }
}
```

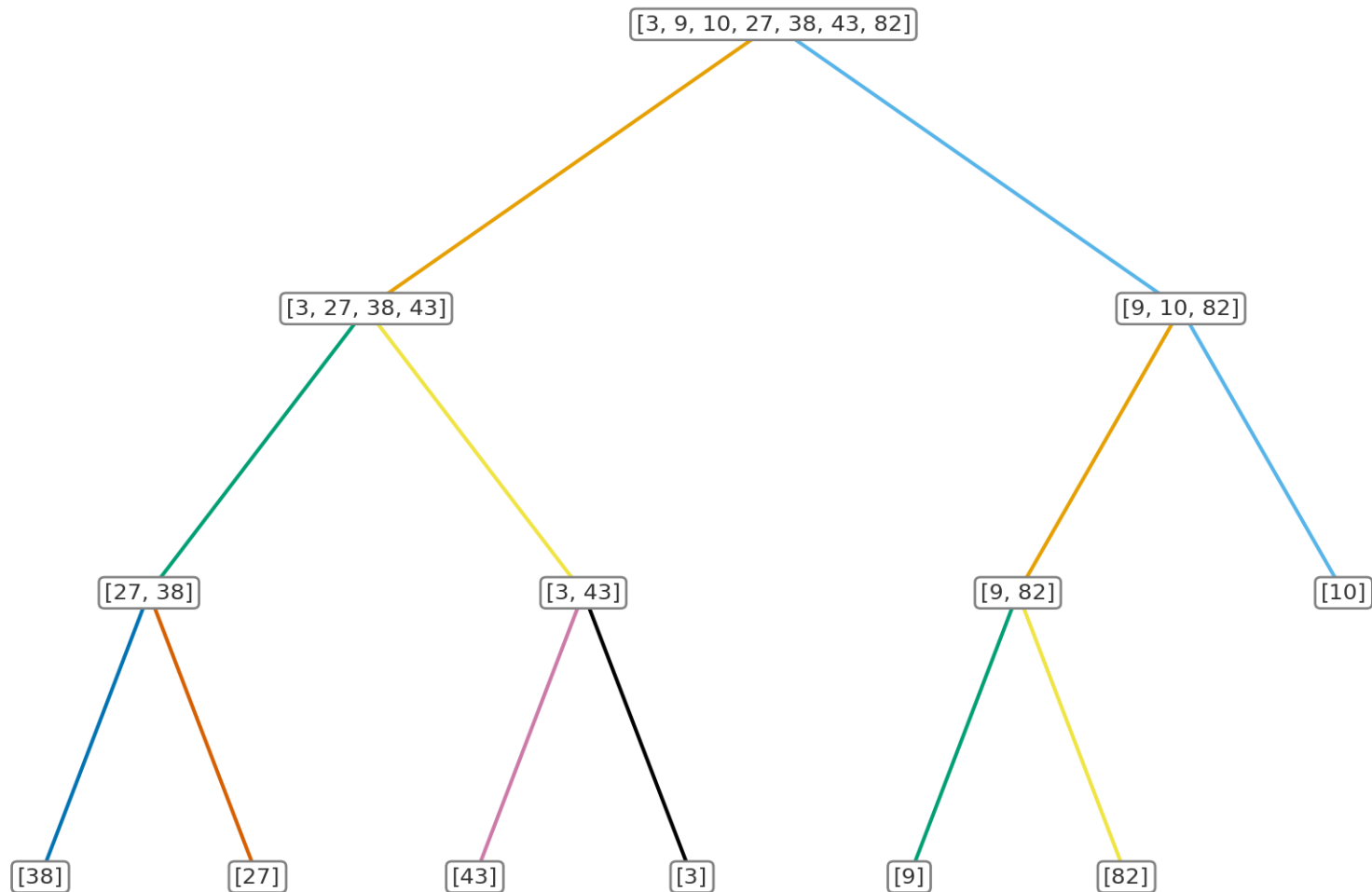
4.5. Sắp xếp trộn

Merge Sort – Quá trình Chia (Divide) (Root ở trên)



4.5. Sắp xếp trộn

Merge Sort – Quá trình Trộn (Merge) (Root ở dưới)



4.6. So sánh các thuật toán

Thuật toán	Trung bình	Xấu nhất	Bộ nhớ	Đặc điểm
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Thực tế nhanh, không ổn định
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Ổn định, cần mảng phụ
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	Không ổn định, dùng trong hệ thống cần giới hạn bộ nhớ